

Kanban — Multi-Agent Profile Collaboration

Want a walkthrough? Read the [Kanban tutorial](#) — four user stories (solo dev, fleet farming, role pipeline with retry, circuit breaker) with dashboard screenshots of each. This page is the reference; the tutorial is the narrative.

Hermes Kanban is a durable task board, shared across all your Hermes profiles, that lets multiple named agents collaborate on work without fragile in-process subagent swarms. Every task is a row in `~/ .hermes/kanban.db`; every handoff is a row anyone can read and write; every worker is a full OS process with its own identity.

Two surfaces: the model talks through tools, you talk through the CLI

The board has two front doors, both backed by the same `~/ .hermes/kanban.db`:

- **Agents drive the board through a dedicated `kanban_*` toolset** — `kanban_show`, `kanban_list`, `kanban_complete`, `kanban_request_review`, `kanban_request_changes`, `kanban_block`, `kanban_heartbeat`, `kanban_comment`, `kanban_attach`, `kanban_attach_url`, `kanban_attachments`, `kanban_create`, `kanban_link`, `kanban_unblock`. The dispatcher spawns each worker with these tools already in its schema; orchestrator profiles can also enable the kanban toolset explicitly. The model reads and routes tasks by calling tools directly, *not* by shelling out to `hermes kanban`. See How workers interact with the board below.
- **You (and scripts, and cron) drive the board through `hermes kanban`** ... on the CLI, `/kanban ...` as a slash command, or the dashboard. These are for humans and automation — the places without a tool-calling model behind them.

Both surfaces route through the same `kanban_db` layer, so reads see a consistent view and writes can't drift. The rest of this page shows CLI examples because they're easy to copy-paste, but every CLI verb has a tool-call equivalent the model uses.

This is the shape that covers the workloads `delegate_task` can't:

- **Research triage** — parallel researchers + analyst + writer, human-in-the-loop.
- **Scheduled ops** — recurring daily briefs that build a journal over weeks.
- **Digital twins** — persistent named assistants (`inbox-triage`, `ops-review`) that accumulate memory over time.
- **Engineering pipelines** — `decompose` → `implement` in parallel

- **Engineering pipelines** — decompose / implement in parallel worktrees → review → iterate → PR.
- **Fleet work** — one specialist managing N subjects (50 social accounts, 12 monitored services).

For the full design rationale, comparative analysis against Cline Kanban / Paperclip / NanoClaw / Google Gemini Enterprise, and the eight canonical collaboration patterns, see docs/hermes-kanban-v1-spec.pdf in the repository.

Kanban vs. delegate_task

They look similar; they are not the same primitive.

	delegate_task	Kanban
Shape	RPC call (fork → join)	Durable message queue + state machine
Parent	Blocks until child returns	Fire-and-forget after create
Child identity	Anonymous subagent	Named profile with persistent memory
Resumability	None — failed = failed	Block → unblock → re-run; crash → reclaim
Human in the loop	Not supported	Comment / unblock at any point
Agents per task	One call = one subagent	N agents over task's life (retry, review, follow-up)
Audit trail	Lost on context compression	Durable rows in SQLite forever
Coordination	Hierarchical (caller → callee)	Peer — any profile reads/writes any task

One-sentence distinction: delegate_task is a function call; Kanban is a work queue where every handoff is a row any profile (or human) can see and edit.

Use delegate_task when the parent agent needs a short reasoning answer before continuing, no humans involved, result goes back into the parent's context.

Use Kanban when work crosses agent boundaries, needs to survive restarts, might need human input, might be picked up by a different role, or needs to be discoverable after the fact.

They coexist: a kanban worker may call delegate_task internally during its run.

Core concepts

- **Board** — a standalone queue of tasks with its own SQLite DB, workspaces directory, and dispatcher loop. A single install can have many boards (e.g. one per project, repo, or domain); see Boards (multi-project) below. Single-project users stay on the default board

and never see the word "board" outside this docs section.

- **Task** — a row with title, optional body, one assignee (a profile name), status (triage | todo | ready | running | blocked | review | done | archived), optional tenant namespace, optional idempotency key (dedup for retried automation).
- **Link** — task_links row recording a parent → child dependency. The dispatcher promotes todo → ready when all parents are done.
- **Comment** — the inter-agent protocol. Agents and humans append comments; when a worker is (re-)spawned it reads the full comment thread as part of its context.
- **Workspace** — the directory a worker operates in. Three kinds:
 - scratch (default) — fresh tmp dir under
~/.hermes/kanban/workspaces/<id>/ (or
~/.hermes/kanban/boards/<slug>/workspaces/<id>/ on non-default boards). **Deleted when the task completes** — scratch is ephemeral by design. Files explicitly declared through kanban_complete(artifacts=[...]) are copied into durable per-task attachment storage before cleanup; existing deliverable paths in legacy completion summaries receive the same treatment. Other scratch files are removed. A missing declared scratch artifact keeps the task in-flight so the worker can correct the path and retry. Use worktree: or dir:<path> when the whole workspace should remain available. The first time a scratch workspace is created on an install, the dispatcher logs a warning and emits a tip_scratch_workspace event on the task (visible via hermes kanban show <id>).
 - dir:<path> — an existing shared directory (Obsidian vault, mail ops dir, per-account folder). **Must be an absolute path**. Relative paths like dir:../tenants/foo/ are rejected at dispatch because they'd resolve against whatever CWD the dispatcher happens to be in, which is ambiguous and a confused-deputy escape vector. The path is otherwise trusted — it's your box, your filesystem, the worker runs with your uid. This is the trusted-local-user threat model; kanban is single-host by design.
Preserved on completion.
 - worktree — a git worktree under .worktrees/<id>/ for coding tasks. Use worktree:<path> to pin the exact target path. Worker-side git worktree add creates it, using --branch when provided.

Preserved on completion.

- **Dispatcher** — a long-lived loop that, every N seconds (default 60): reclaims stale claims, reclaims crashed workers (PID gone but TTL not yet expired), promotes ready tasks, atomically claims, spawns assigned profiles. Runs **inside the gateway** by default (kanban.dispatch_in_gateway: true). One dispatcher sweeps all boards per tick; workers are spawned with HERMES_KANBAN_BOARD pinned so they can't see other boards. After kanban.failure_limit consecutive spawn failures on the same task (default: 2) the dispatcher auto-blocks it with the last error as the reason — prevents

dispatcher auto-blocks it with the last error as the reason — prevents thrashing on tasks whose profile doesn't exist, workspace can't mount, etc.

- **Tenant** — optional string namespace *within* a board. One specialist fleet can serve multiple businesses (`--tenant business-a`) with data isolation by workspace path and memory key prefix. Tenants are a soft filter; boards are the hard isolation boundary.

Boards (multi-project)

Boards let you separate unrelated streams of work — one per project, repo, or domain — into isolated queues. A new install has exactly one board called `default` (DB at `~/hermes/kanban.db` for back-compat). Users who only want one stream of work never need to know about boards; the feature is opt-in.

Per-board isolation is absolute:

- Separate SQLite DB per board
(`~/hermes/kanban/boards/<slug>/kanban.db`).
- Separate `workspaces/` and `logs/` directories.
- Workers spawned for a task see **only** their board's tasks — the dispatcher sets `HERMES_KANBAN_BOARD` in the child env and every `kanban_*` tool the worker has access to reads it.
- Linking tasks across boards is not allowed (keeps the schema simple; if you really need cross-project refs, use free-text mentions and look them up by id manually).

Managing boards from the CLI

Board resolution order (highest precedence first):

1. Explicit `--board <slug>` on the CLI call.
2. `HERMES_KANBAN_BOARD` env var (set by the dispatcher when spawning a worker, so workers can't see other boards).
3. `~/hermes/kanban/current` — the slug persisted by `hermes kanban boards switch`.
4. `default`.

Slugs are validated: lowercase alphanumerics + hyphens + underscores, 1-64 chars, must start with alphanumeric. Uppercase input is auto-downcased. Anything else (slashes, spaces, dots, ..) is rejected at the CLI layer so path-traversal tricks can't name a board.

Managing boards from the dashboard

`hermes dashboard` → Kanban tab shows a board switcher at the top as soon as more than one board exists (or any board has tasks). Single-board users see only a small + New board button; the switcher is hidden until it matters.

- **Board dropdown** — pick the active board. Your selection is saved to

the browser's `localStorage` so it persists across reloads without shifting the CLI's current pointer out from under a terminal you left open.

- **+ New board** — opens a modal asking for slug, display name, description, and icon. Option to auto-switch to the new board.
- **Settings** — opens a modal for editing the current board's display name, description, and **project directory** (`default_workdir`). The project directory is the board-level workspace default every new task inherits (git repo → preserved worktree, plain dir → preserved directory); each task can still override it at creation time. Clearing the field reverts new tasks to disposable scratch workspaces.
- **Archive** — only shown on non-default boards. Confirms, then moves the board dir to `boards/_archived/`.

All dashboard API endpoints accept `?board=<slug>` for board scoping. The events WebSocket is pinned to a board at connection time; switching in the UI opens a fresh WS against the new board.

File attachments

Tasks can carry file attachments — PDFs, images, source documents — so a worker has the source material it needs without you pasting paths into the body and hoping it finds them.

- **Upload** — open a task in the dashboard drawer and use the **Attachments** section's *Upload file* button (multiple files at once are fine). Each upload is capped at 25 MB.
- **Storage** — files land under `<hermes-home>/kanban/attachments/<task_id>/` for the default board, or `<hermes-home>/kanban/boards/<slug>/attachments/<task_id>/` for a named board. Set `HERMES_KANBAN_ATTACHMENTS_ROOT` to pin a custom location.
- **What the worker sees** — when the dispatcher hands a task to a worker, the worker's context includes an **Attachments** section listing each file's name and its **absolute path**. The worker has full file/terminal tool access, so it reads attachments directly (`read_file`, or shell tools like `pdftotext`).
- **Download / remove** — the drawer lists each attachment with a download link and a remove (x) control. Removing an attachment deletes both the metadata row and the on-disk file.

Remote terminal backends

Attachment paths resolve directly on the **local** terminal backend, which is

the default for Kanban workers. If you run workers on a remote backend (Docker, Modal), mount the board's `attachments/` directory into the sandbox so the absolute paths in the worker context are reachable.

Quick start

The commands below are **you** (the human) setting up the board and creating tasks. Once a task is assigned, the dispatcher spawns the assigned profile as a worker, and from there **the model drives the task through kanban_* tool calls, not CLI commands** — see How workers interact with the board.

When the dispatcher picks up `t_abcd` and spawns the researcher profile, the very first thing that worker's model does is call `kanban_show()` to read its task. It doesn't run `hermes kanban show t_abcd`.

Gateway-embedded dispatcher (default)

The dispatcher runs inside the gateway process. Nothing to install, no separate service to manage — if the gateway is up, ready tasks get picked up on the next tick (60s by default).

Override the config flag at runtime via

`HERMES_KANBAN_DISPATCH_IN_GATEWAY=0` for debugging. Standard gateway supervision applies: run `hermes gateway start` directly, or wire the gateway up as a systemd user unit (see the gateway docs). Without a running gateway, ready tasks stay where they are until one comes up — `hermes kanban create` warns about this at creation time.

Running `hermes kanban daemon` as a separate process is **deprecated**; use the gateway. If you truly cannot run the gateway (headless host policy forbids long-lived services, etc.) a `--force` escape hatch keeps the old standalone daemon alive for one release cycle, but running both a gateway-embedded dispatcher AND a standalone daemon against the same `kanban.db` causes claim races and is not supported.

Idempotent create (for automation / webhooks)

Bulk CLI verbs

All the lifecycle verbs accept multiple ids so you can clean up a batch in one command:

Where an unblocked task lands

`unlock` restores the safe source phase: **review** for reviewer-origin work whose parents are complete, **ready** for implementation work whose parents are complete, or **todo** while any parent remains open. A `todo` task keeps its source-phase provenance and returns to `review` or `ready` automatically when the dependency gate clears. `unlock` never routes directly to `triage`.

If you `unlock` a task and it later shows up in **triage**, the `unlock` is not what put it there. A subsequent *re-block for the same reason* did: after a task is blocked → unblocked → re-blocked for the same cause `BLOCK_RECURRENCE_LIMIT` times (default 2), the `unlock-loop` breaker stops sending it back to `triage` — where a cron would just keep unblocking it

sending it back to blocked — where a cron would just keep unblocking it — and routes it to triage for a human decision. This is a deterministic DB guard, not an LLM judgment call, and a task's body text cannot opt out of it: the recurrence counter deliberately survives each unblock (it resets only on a successful complete). To keep an unblocked task in the work pool, resolve *why it keeps re-blocking* (unfinished parent, missing input, unmet capability) before unblocking, or raise BLOCK_RECURRENCE_LIMIT if the loop is expected.

How workers interact with the board

Workers do not shell out to hermes kanban. When the dispatcher spawns a worker it sets HERMES_KANBAN_TASK=t_abcd in the child's env, and that env var flips on a dedicated **kanban toolset** in the model's schema. The same toolset is also available to orchestrator profiles that enable kanban in their toolsets config. These tools read and mutate the board directly via the Python kanban_db layer, same as the CLI does. A running worker calls these like any other tool; it never sees or needs the hermes kanban CLI.

Tool	Purpose	Required params
kanban_show	Read the current task (title, body, prior attempts, parent handoffs, comments, full pre-formatted worker_context). Defaults to the env's task id.	—
kanban_list	List task summaries with filters for assignee, status, tenant, archived visibility, and limit. Intended for orchestrators discovering board work.	—
kanban_complete	Finish with summary + metadata structured handoff.	at least one of summary / result
kanban_request_review	Start same-card review with a durable summary, optional metadata, and optional reviewer profile. The task moves to review; this is not a block.	summary
kanban_request_changes	Reviewer verdict from an active review run. Closes that run, reapplies parent gating, and routes the task to its original implementer without block-loop accounting.	reason
kanban_block	Stop work and route by why: kind=dependency (waits in todo, auto-resumes), needs_input/capability/transient (surface to a human). Repeated same-kind re-blocks auto-escalate to triage.	reason
kanban_heartbeat	Signal liveness during long operations. Pure side-effect.	—
kanban_comment	Append a durable note to the task thread.	task_id, body
kanban_attach	Attach a file to a task by passing its bytes inline (base64); stored under the task's attachments dir (25 MB cap).	file bytes + name

`kanban_complete(summary=..., metadata={...})` is intentionally flexible: the summary is the human-readable closeout, and metadata is the machine-readable handoff that downstream agents, reviewers, or dashboards can reuse without scraping prose.

For engineering and review tasks, prefer this optional metadata shape:

These keys are a convention, not a schema requirement. The useful property is that every worker leaves enough evidence for the next reader to answer four questions quickly:

1. What changed?
2. How was it verified?
3. What can unblock or retry this if it fails?
4. What risk is still deliberately left open?

Keep secrets, raw logs, tokens, OAuth material, and unrelated transcripts out of metadata. Store pointers and summaries instead. If a task has no files or tests, say so explicitly in summary and use metadata for the evidence that does exist, such as source URLs, issue ids, or manual review steps.

The worker lifecycle

Every profile that works kanban tasks automatically gets the worker lifecycle — it's injected into the worker's system prompt at spawn (the `KANBAN_GUIDANCE` block), so there is **nothing to install or configure**. It teaches the worker the full lifecycle in **tool calls**, not CLI commands:

1. On spawn, call `kanban_show()` to read title + body + parent handoffs + prior attempts + full comment thread.
2. `cd $HERMES_KANBAN_WORKSPACE` (via the terminal tool) and do the work there.
3. Call `kanban_heartbeat(note="...")` every few minutes during long operations. **If your work may run longer than 1 hour, call `kanban_heartbeat` at least once an hour** — the dispatcher reclaims tasks that have been running past `kanban.dispatch_stale_timeout_seconds` (default 4 h) with no heartbeat in the last hour, on the assumption the worker crashed without cleanup. A reclaim is benign (the task goes back to ready for re-dispatch without a failure-counter tick) but you lose your current run's progress.
4. Complete with `kanban_complete(summary="...", metadata={...})`, or `kanban_block(reason="...")` if stuck.

That final `kanban_complete` / `kanban_block` call is part of the worker protocol. If the worker process exits with status 0 while the task is still running, the dispatcher treats that as a protocol violation and emits a `protocol_violation` event.

Agent-side prevention: Before the worker exits, Hermes injects up to two synthetic nudges when it detects the model is about to stop without a

terminal board tool call. This catches the common case where the model narrates the next step ("Let me write the report") and stops with `finish_reason=stop`. The nudge reminds the model to call `kanban_complete` or `kanban_block` immediately. This guard is active only for

dispatcher-spawned workers (`HERMES_KANBAN_TASK` is set) and can be disabled with `HERMES_KANBAN_STOP_NUDGE=0`.

Dispatcher-side recovery: If the nudges are exhausted or the worker crashes before reaching the nudge, the dispatcher gives the violation a **bounded retry** (up to `_PROTOCOL_VIOLATION_FAILURE_LIMIT` consecutive violations, default 3) before auto-blocking the task instead of respawning it into the same loop. The budget counts only *consecutive* clean-exit protocol violations — interleaved rate-limited requeues are neutral, and any other failure kind resets the streak — and a per-task `max_retries` overrides the bound. This usually means the model wrote a plain-text answer and exited without using the Kanban tool surface.

The lifecycle plus the load-bearing reference details (workspace kinds, deliverable artifacts, claiming created cards) ship in that system-prompt block, so every worker has them regardless of which profile it runs under — no per-profile skill setup required.

Pinning extra skills to a specific task

Sometimes a single task needs specialist context the assignee profile doesn't carry by default — a translation job that needs the `translation` skill, a review task that needs `github-code-review`, a security audit that needs `security-pr-audit`. Rather than editing the assignee's profile every time, attach the skills directly to the task.

From an orchestrator agent (the usual case — one agent routing work to another), use the `kanban_create` tool's `skills` array:

From a human (CLI / slash command), repeat `--skill` for each one:

From the dashboard, type the skills comma-separated into the **skills** field of the create-task dialog.

The dispatcher emits one `--skills <name>` flag per skill listed, so the worker spawns with all of them loaded on top of the auto-injected kanban guidance. The skill names must match skills that are actually installed on the assignee's profile (run `hermes skills list` to see what's available); there's no runtime install.

Per-task model override

Pin a task's worker to a specific model (and optionally provider), independent of the assignee profile's default:

The dispatcher spawns the worker with the pinned model (`--provider`

<name> is passed when set; --provider requires a model). The dashboard's per-task model dropdown drives the same model_override field. With no override, the worker uses its profile's configured model.

Cost strategy: frontier orchestrator, inexpensive workers

Kanban's per-profile configs make the planner/worker cost split natural. Decomposing a project into well-scoped cards takes frontier-level judgment; executing a card that already carries a clear goal, context, and handoff evidence usually doesn't — and the workers are where the vast majority of tokens are spent, so the worker model is where the cost lives. Run your orchestrator/dispatcher profile on a frontier model and point worker profiles at inexpensive models. Each profile has its own config.yaml under ~/.hermes/profiles/<name>/, and the dispatcher injects the profile-scoped HERMES_HOME when it spawns hermes -p <assignee>, so each worker reads its own profile's model settings:

For the occasional quality-sensitive card, pin just that task back to a stronger model with the per-task model override (--model/--provider at create time, hermes kanban set-model later, or the dashboard's model dropdown) — no profile edits needed.

Lifecycle plugin hooks

Board transitions fire [plugin hooks](#): kanban_task_claimed,

kanban_task_completed, and kanban_task_blocked, each carrying task_id and profile_name. Hooks fire **after** the board DB change commits, so callbacks always see durable state. Note the process split: kanban_task_claimed fires in the **dispatcher** process, while kanban_task_completed/kanban_task_blocked fire in the **worker** process — register the hook in the dispatcher profile to observe every transition centrally.

Goal-mode cards (--goal)

By default each worker gets **one shot** at its card — do the work, call kanban_complete/kanban_block, exit. Pass --goal (CLI) or goal_mode=True (the kanban_create tool / dashboard) to instead run that worker in a **goal loop**, the same Ralph-style engine behind the /goal slash command: after every turn an auxiliary judge checks the worker's output against the card's title + body (treated as the acceptance criteria), and if the work isn't done — and the turn budget remains — the worker keeps going **in the same session** until the judge agrees, the worker terminates the task itself, or the budget runs out (which **blocks** the card for human review rather than exiting silently).

Use it for open-ended, multi-step, or "keep going until X is true" cards. Skip it for cheap one-shot work — the per-turn judge overhead isn't worth it, and the dispatcher's existing retry/circuit-breaker already handles transient worker failures. The judge is only as good as your goal text, so

transient worker failures. The judge is only as good as your text, so write the body as **explicit acceptance criteria**.

Goal-mode cards borrow the `/goal` engine — they don't connect to it

--`goal` runs the continuation loop *inside that one card's worker session*. It shares the engine with the [/goal slash command](#), not the state: setting a `/goal` in a chat session never creates, claims, or moves a kanban card, and a goal-mode card's loop is invisible to any chat session's `/goal` status. If you want this conversation to keep iterating, use [/goal](#); if you want work on the board, create a card.

How the orchestrator behaves

A **well-behaved orchestrator does not do the work itself**. It decomposes the user's goal into tasks, links them, assigns each to one of the profiles you've set up, and steps back. The orchestrator guidance — anti-temptation rules, a Step-0 profile-discovery prompt (the dispatcher silently fails on unknown assignee names, so the orchestrator must ground every card in profiles that actually exist on your machine), and a decomposition playbook keyed on `kanban_create` / `kanban_link` / `kanban_comment` — is injected into the worker's system prompt automatically; there is nothing to install.

A canonical orchestrator turn (two parallel researchers handing off to a writer):

The orchestrator guidance ships in the worker's system prompt automatically — there is nothing to install or sync per profile.

Decide before you fan out. Design decisions belong to the orchestrator, not to the workers. If two parallel cards would each have to pick the same thing — a naming scheme, a schema, a file format, an API shape — the orchestrator decides it once and stamps the decision into **both** card bodies. Workers cannot see sibling cards, so every child card body must carry every decision it depends on. Example: for the parallel cards "build the exporter" and "build the importer", don't let each worker invent its own file format — pick one up front (say, newline-delimited JSON with a `version` field) and write it into both bodies, or the two halves will never round-trip.

For best results, pair it with a profile whose toolsets are restricted to board operations (`kanban`, `gateway`, `memory`) so the orchestrator literally cannot execute implementation tasks even if it tries.

Dashboard (GUI)

The `/kanban` CLI and slash command are enough to run the board headlessly, but a visual board is often the right interface for humans-in-the-loop: triage, cross-profile supervision, reading comment threads, and dragging cards between columns. Hermes ships this as a **bundled dashboard plugin** at `plugins/kanban/` — not a core feature, not a separate service — following the model laid out in [Extending the](#)

separate service — following the model laid out in [Extending the Dashboard](#).

Open it with:

What the plugin gives you

- A **Kanban** tab showing one column per status: triage, todo, ready, running, blocked, done (plus archived when the toggle is on).
 - triage is the parking column for rough ideas. By default (`kanban.auto_decompose: true`), the dispatcher auto-runs the **decomposer** on tasks that land here. The built-in decomposer uses the `auxiliary.kanban_decomposer` model path, reads your profile roster (with descriptions), and fans the task out into a small graph of child tasks routed to the best-fit specialists. The original task stays alive as the parent of every child so its assignee (`kanban.orchestrator_profile`, or the active default profile when unset) wakes back up to judge completion when everything finishes. Flip the **Orchestration: Auto/Manual** pill at the top of the page (emerald = Auto, muted gray = Manual), or by editing `config.yaml` directly. Both modes coexist with `hermes kanban specify` — that's still available as a single-task spec rewrite when you don't want fan-out.
- Cards show the task id, title, priority badge, tenant tag, assigned profile, comment/link counts, a **progress pill** (N/M children done when the task has dependents), and "created N ago". A per-card checkbox enables multi-select.
- **Per-profile lanes inside Running** — toolbar checkbox toggles sub-grouping of the Running column by assignee.
- **Live updates via WebSocket** — the plugin tails the append-only

`task_events` table on a short poll interval; the board reflects changes the instant any profile (CLI, gateway, or another dashboard tab) acts. Reloads are debounced so a burst of events triggers a single refetch.

- **Drag-drop** cards between columns to change status. The drop sends `PATCH /api/plugins/kanban/tasks/:id` which routes through the same `kanban_db` code the CLI uses — the three surfaces can never drift. Moves into destructive statuses (done, archived, blocked) prompt for confirmation. Touch devices use a pointer-based fallback so the board is usable from a tablet.
- **Create-task dialog** — click + on any column header to open a modal with labeled fields: title, assignee, priority, skills, workspace kind/path (seeded from the board's project directory; per-task override), goal mode, and (optionally) a parent task from a dropdown over every existing task. Press Enter to create the task, Shift+Enter to insert a newline in the title field, or Escape to cancel. Creating from the Triage column automatically parks the new task in triage.
- **Multi-select with bulk actions** — shift/ctrl-click a card or tick its checkbox to add it to the selection. A bulk action bar appears at the top with batch status transitions, archive, and reassign (by profile

dropdown, or "(unassign)"). Destructive batches confirm first. Per-id partial failures are reported without aborting the rest.

- **Click a card** (without shift/ctrl) to open a side drawer (Escape or click-outside closes) with:
 - **Editable title** — click the heading to rename.
 - **Editable assignee / priority** — click the meta row to rewrite.
 - **Editable description** — markdown-rendered by default (headings, bold, italic, inline code, fenced code, `http(s) / mailto:` links, bullet lists), with an "edit" button that swaps in a textarea. Markdown rendering is a tiny, XSS-safe renderer — every substitution runs on HTML-escaped input, only `http(s) / mailto:` links pass through, and `target="_blank" + rel="noopener noreferrer"` are always set.
 - **Dependency editor** — chip list of parents and children, each with an `x` to unlink, plus dropdowns over every other task to add a new parent or child. Cycle attempts are rejected server-side with a clear message.
 - **Status action row** (`→ triage / → ready / → running / block / unblock / complete / archive`) with confirm prompts for destructive transitions. For cards in the **Triage** column the row also exposes two LLM-driven actions: 🧠 **Decompose** fans the task out into a graph of child tasks routed to specialist profiles by description, and ✨ **Specify** does a single-task spec rewrite. Decompose falls back to specify-style promotion when the LLM decides the task doesn't benefit from fan-out, so it's a strict superset. Both are reachable from the CLI (`hermes kanban decompose <id> / specify <id> / --all`), from any gateway platform (`/kanban decompose <id>`), and programmatically via `POST /api/plugins/kanban/tasks/:id/decompose` and `.../specify`. Configure the models under `auxiliary.kanban_decomposer` and `auxiliary.triage_specifier` in `config.yaml`.
 - Result section (also markdown-rendered), comment thread with Enter-to-submit, the last 20 events.
- **Toolbar filters** — free-text search, tenant dropdown (defaults to `dashboard.kanban.default_tenant` from `config.yaml`), assignee dropdown, "show archived" toggle, "lanes by profile" toggle, and a **Nudge dispatcher** button so you don't have to wait for the next 60 s tick.

Visually the target is the familiar Linear / Fusion layout: dark theme, column headers with counts, coloured status dots, pill chips for priority and tenant. The plugin reads only theme CSS vars (`--color-*`, `--radius`, `--font-mono`, ...), so it reskins automatically with whichever dashboard theme is active.

Auto vs Manual orchestration

The kanban board has two ways to handle a task you drop into the Triage column:

Auto (default) — `kanban.auto_decompose: true`. The gateway-embedded dispatcher runs the **decomposer** on each tick, capped by `kanban.auto_decompose_per_tick` (default 3 tasks per tick) so a bulk-load of triage tasks doesn't burst-spend the auxiliary LLM. The decomposer uses the built-in decomposition prompt plus the `auxiliary.kanban_decomposer` model path, reads your installed profiles + their descriptions, and asks the LLM to produce a JSON task graph: which tasks to spawn, who they go to, and which depend on which. The original triage task becomes the parent of every leaf in the graph, so it stays alive until the whole graph completes - and then promotes back to ready so its assignee (`kanban.orchestrator_profile`, or the active default profile when unset) can judge completion and add more tasks if the work isn't done. This is the "drop a one-liner, walk away" flow.

Manual — `kanban.auto_decompose: false`. Triage tasks stay in triage until you act. Click the  **Decompose** button on a card, run `hermes kanban decompose <id>` (or `--all`), or use `/kanban decompose <id>` from a chat. This matches the pre-decomposer behavior of the board, useful when you want full control over what runs when.

Flip between the two modes from the **Orchestration: Auto/Manual** pill at the top of the kanban page (emerald = Auto, muted gray = Manual), or by editing `config.yaml` directly. Both modes coexist with `hermes kanban specify` — that's still available as a single-task spec rewrite when you don't want fan-out.

The decomposer's routing decisions depend on profile descriptions, which is a per-profile labeling primitive you set with `hermes profile create --description "...", hermes profile describe <name> --text "...", hermes profile describe <name> --auto` (LLM-generates from the profile's installed skills + model), or the dashboard's per-profile editor in the expanded **Orchestration settings** panel. Profiles without a description still appear in the roster — they're routable by name, just less precisely. The decomposer NEVER lands a child task with `assignee=None`:

when the LLM picks an unknown profile, the child gets routed to `kanban.default_assignee` (or the active default profile if that's unset).

`kanban.orchestrator_profile` does not load that profile's prompt, skills, or custom logic into the decomposition call. It controls who owns the root/orchestration task after fan-out. To change the decomposer's model/provider, configure `auxiliary.kanban_decomposer`. To use a profile's custom task-splitting logic instead of the built-in decomposer, switch to Manual mode and have that profile create or decompose tasks explicitly.

Config knobs (all under `kanban:` in `~/hermes/config.yaml`):

Key	Default	Purpose
auto_decompose	true	Dispatcher auto-runs the decomposer every tick.

auto_decompose_per_tick	3	Cap on decompositions per dispatcher tick. Excess defers to the next tick.
orchestrator_profile	""	Profile assigned to the root/orchestration task after decomposition. Empty = fall back to active default profile.
default_assignee	""	Where a child task lands when the LLM picks an unknown profile. Empty = fall back to active default.
auto_subscribe_on_create	true	When a worker calls kanban_create from inside a session with a persistent delivery channel (messaging gateway or TUI), the originating session is auto-subscribed to the new task's completion/block events. The dispatcher still drives the delivery — this only changes whether the caller's chat/key shows up in the notify-sub table. Set to false to require explicit kanban_notify-subscribe calls per task.

And the two auxiliary LLM slots:

Key	Purpose
auxiliary.kanban_decomposer	Model that produces the task graph (called by Decompose). Set provider/model to override the main chat model.
auxiliary.profile_describer	Model that auto-generates profile descriptions (called by hermes profile describe --auto).

Architecture

The GUI is strictly a **read-through-the-DB + write-through-kanban_db** layer with no domain logic of its own:

REST surface

All routes are mounted under `/api/plugins/kanban/` and protected by the dashboard's ephemeral session token:

Method	Path	Purpose
GET	<code>/board?tenant=<name>&include_archived=...</code>	Full board grouped by status column, plus tenants + assignees for filter dropdowns
GET	<code>/tasks/:id</code>	Task + comments + events + links
POST	<code>/tasks</code>	Create (wraps <code>kanban_db.create_task</code> , accepts <code>triage: bool</code> and <code>parents: [id, ...]</code>)
PATCH	<code>/tasks/:id</code>	Status / assignee / priority / title / body / result
POST	<code>/tasks/bulk</code>	Apply the same patch (status / archive / assignee / priority) to every id in <code>ids</code> . Per-id failures reported without aborting siblings
POST	<code>/tasks/:id/comments</code>	Append a comment
		Run the triage specifier — auxiliary LLM fleshes out the task body and

POST	/tasks/:id/specify	LLM reshapes out the task body and promotes it from triage to todo. Returns {ok, task_id, reason, new_title}; ok=false with a human-readable reason on "not in triage" / no aux client / LLM error is a 200, not a 4xx
POST	/tasks/:id/decompose	Run the kanban decomposer — auxiliary LLM produces a task graph and the helper atomically creates the children + links the root + flips triage → todo. Returns {ok, task_id, reason, fanout, child_ids, new_title}. Same 200-on-LLM-error convention as /specify.
GET	/profiles	List installed profiles with their descriptions (consumed by the dashboard's profile-description editor and the orchestrator picker).
PATCH	/profiles/:name	Set or clear a profile's description (user-authored — description_auto: false). Returns {ok, profile, description}.
POST	/profiles/:name/describe-auto	Generate a description for a profile via auxiliary.profile_describer. Persists with description_auto: true so the dashboard can surface a "review" badge.
GET	/orchestration	Read the kanban orchestration settings (orchestrator_profile, default_assignee, auto_decompose) plus the <i>resolved</i> effective values after fallbacks.
PUT	/orchestration	Update one or more of the three orchestration keys in config.yaml. Validates that non-empty profile names actually exist.
POST	/links	Add a dependency (parent_id → child_id)
DELETE	/links?parent_id=...&child_id=...	Remove a dependency
POST	/dispatch?max=...&dry_run=...	Nudge the dispatcher — skip the 60 s wait
GET	/config	Read dashboard.kanban preferences from config.yaml — default_tenant, lane_by_profile, include_archived_by_default, render_markdown
WS	/events?since=<event_id>	Live stream of task_events rows

Every handler is a thin wrapper — the plugin is ~700 lines of Python (router + WebSocket tail + bulk batcher + config reader) and adds no new business logic. A tiny `_conn()` helper auto-initializes `kanban.db` on every read and write, so a fresh install works whether the user opened the dashboard first, hit the REST API directly, or ran `hermes kanban init`.

Dashboard config

Any of these keys under `dashboard.kanban` in `~/hermes/config.yaml`

changes the tab's defaults — the plugin reads them at load time via `GET /config`:

Each key is optional and falls back to the shown default.

Security model

The dashboard's HTTP auth middleware [explicitly skips /api/plugins/](#) — plugin routes are unauthenticated by design because the dashboard binds to localhost by default. That means the kanban REST surface is reachable from any process on the host.

The WebSocket takes one additional step: it requires the dashboard's ephemeral session token as a `?token=...` query parameter (browsers can't set `Authorization` on an upgrade request), matching the pattern used by the in-browser PTY bridge.

If you run `hermes dashboard --host 0.0.0.0`, every plugin route — kanban included — becomes reachable from the network. **Don't do that on a shared host.** The board contains task bodies, comments, and workspace paths; an attacker reaching these routes gets read access to your entire collaboration surface and can also create / reassign / archive tasks.

Tasks in `~/hermes/kanban.db` are profile-agnostic on purpose (that's the coordination primitive). If you open the dashboard with `hermes -p <profile> dashboard`, the board still shows tasks created by any other profile on the host. Same user owns all profiles, but this is worth knowing if multiple personas coexist.

Live updates

`task_events` is an append-only SQLite table with a monotonic `id`. The WebSocket endpoint holds each client's last-seen event id and pushes new rows as they land. When a burst of events arrives, the frontend reloads the (very cheap) board endpoint — simpler and more correct than trying to patch local state from every event kind. WAL mode means the read loop never blocks the dispatcher's `BEGIN IMMEDIATE` claim transactions.

Extending it

The plugin uses the standard Hermes dashboard plugin contract — see [Extending the Dashboard](#) for the full manifest reference, shell slots, page-scoped slots, and the Plugin SDK. Extra columns, custom card chrome, tenant-filtered layouts, or full `tab.override` replacements are all expressible without forking this plugin.

To disable without removing: add `dashboard.plugins.kanban.enabled: false` to `config.yaml` (or delete `plugins/kanban/dashboard/manifest.json`).

Scope boundary

The GUI is deliberately thin. Everything the plugin does is reachable from the CLI; the plugin just makes it comfortable for humans. Auto-assignment, budgets, governance gates, and org-chart views remain user-space — a router profile, another plugin, or a reuse of `tools/approval.py` — exactly as listed in the out-of-scope section of the design spec.

CLI command reference

This is the surface **you** (or scripts, cron, the dashboard) use to drive the board. Workers running inside the dispatcher use the `kanban_*` tool surface for the same operations — the CLI here and the tools there both route through `kanban_db`, so the two surfaces agree by construction.

All commands are also available as a slash command in the interactive CLI and in the messaging gateway (see `/kanban` slash command below).

`--max-retries` is a per-task circuit-breaker override for the dispatcher. `--max-retries 1` blocks the task on the first non-successful attempt, while `--max-retries 3` allows two retries and blocks on the third failure. Omit it to use `kanban.failure_limit` from `config.yaml`, then the built-in default.

Concurrency, scheduling, and child promotion config

Config key	Default	What it does
<code>kanban.max_in_progress</code>	unset (unlimited)	Caps the number of simultaneously running tasks. When the board already has N running, the dispatcher skips spawning more — useful for slow workers (local LLMs, resource-constrained hosts) so they finish what they have before more pile up and time out. Invalid or below-1 values log a warning and behave as unlimited.
<code>kanban.max_in_progress_per_profile</code>	unset (unlimited)	Per-profile variant of <code>max_in_progress</code> — caps how many tasks any single assignee profile may run concurrently. Useful when one profile is slow or rate-limited but others should keep flowing. Applies alongside the board-wide <code>max_in_progress</code> ; both must allow a spawn for it to proceed.
		After <code>decompose_triage_task()</code> produces children with no

kanban.auto_promote_children	true	parent-blocker dependencies, they're automatically promoted to ready so the dispatcher can pick them up. Set to false to require manual review — children stay in todo until you promote them.
kanban.default_workdir	unset	Board-level default working directory applied to new tasks when neither --workspace nor the task itself overrides it. Per-task workspace: still wins.

Scheduled task starts (scheduled_at)

Set `scheduled_at` on a task to delay dispatch until a specific time. The dispatcher skips ready tasks whose `scheduled_at` is in the future and picks them up on the first tick after that timestamp.

Respawn guard

The dispatcher refuses to re-spawn a ready task when it hit a `quota/auth/429` error on the previous run (`blocker_auth`), or completed a run successfully within the guard window (`recent_success`), or a recent task comment links to a GitHub PR (`active_pr`). This prevents repeat worker storms on the same bug or task while a human catches up. See the `respawn_guarded` row in the event reference.

Drag-to-delete and bulk delete (dashboard)

The dashboard exposes a **trash drop zone** on the kanban page — drag any card into it to delete the task (cascades through `task_events`, child links, and subscriptions). A confirmation prompt protects against accidents. Bulk delete is also reachable via `DELETE /api/plugins/kanban/tasks` with a JSON body `{"ids": ["t_abc", "t_def", ...]}`.

Worker visibility endpoints

The dashboard plugin API now exposes these read-only endpoints (plus a run-control verb) for external monitors:

Endpoint	Returns
GET /api/plugins/kanban/workers/active	Currently spawned workers with PID, profile, task id, started-at, last heartbeat
GET /api/plugins/kanban/runs/{id}	Single-run detail — task id, status, started/ended, exit code, log path
POST /api/plugins/kanban/runs/{id}/stop	Terminate a reclaimable run — stops the worker and frees

<code>/api/plugins/kanban/runs/{run_id}/terminate</code>	Stops the worker and re-dispatches the task for re-dispatch
<code>GET /api/plugins/kanban/inspect</code>	Combined dispatcher snapshot — backlog, in-progress count vs. <code>max_in_progress</code> , recent events

All of these are gated by the same dashboard plugin auth as the rest of the kanban plugin API.

Kanban Swarm topology helper

`hermes kanban swarm` creates a durable **Kanban Swarm v1** graph in one shot: a completed root/blackboard card, N parallel worker cards, a verifier card gated on all workers, and a synthesizer card gated on the verifier. Shared swarm context (the "blackboard") is stored as structured JSON

comments on the root card so any worker can read it.

The resulting graph is committed atomically: dispatchers and dashboard readers see either no new swarm or the complete topology, never a partially linked root/worker/verifier graph. It then dispatches normally — workers run in parallel, the verifier wakes after they all finish, and the synthesizer wakes after the verifier marks the work clean.

/kanban slash command

Every `hermes kanban <action>` verb is also reachable as `/kanban <action>` — from inside an interactive `hermes chat` session **and** from any gateway platform (Telegram, Discord, Slack, WhatsApp, Signal, Matrix, Mattermost, email, SMS). Both surfaces call the exact same `hermes_cli.kanban.run_slash()` entry point that reuses the `hermes kanban` argparse tree, so the argument surface, flags, and output format are identical across CLI, `/kanban`, and `hermes kanban`. You don't have to leave the chat to drive the board.

Quote multi-word arguments the same way you would on a shell — `run_slash` parses the rest of the line with `shlex.split`, so `"..."` and `'...'` both work.

Mid-run usage: /kanban bypasses the running-agent guard

The gateway normally queues slash commands and user messages while an agent is still thinking — that's what stops you from accidentally starting a second turn while the first is in flight. **/kanban is explicitly exempted from this guard.** The board lives in `~/.hermes/kanban.db`, not in the running agent's state, so reads (`list`, `show`, `context`, `tail`, `watch`, `stats`, `runs`) and writes (`comment`, `unblock`, `block`, `assign`, `archive`, `create`, `link`, ...) all go through immediately, even mid-turn.

This is the whole point of the separation:

- A worker blocks waiting on a peer → you send `/kanban unblock t_abcd` from your phone and the dispatcher picks the peer up on its next tick. The blocked worker isn't interrupted — it just stops being blocked.
- You spot a card that needs human context → `/kanban comment t_xyz "use the 2026 schema, not 2025"` lands on the task thread and the *next* run of that task will read it in `kanban_show()`.
- You want to know what your fleet is doing without stopping the orchestrator → `/kanban list --mine` or `/kanban stats` inspects the board without touching your main conversation.

Auto-subscribe on `/kanban create` (gateway only)

When you create a task from the gateway with `/kanban create "..."`, the originating chat (platform + chat id + thread id) is automatically subscribed to that task's terminal events (`completed`, `blocked`, `gave_up`, `crashed`, `timed_out`). You'll get one message back per terminal event — including the first line of the worker's result summary on `completed` — without having to poll or remember the task id.

Subscriptions auto-remove themselves once the task reaches done or archived. If you script a create with `--json` (machine output) the auto-subscribe is skipped — the assumption is that scripted callers want to manage subscriptions explicitly via `/kanban notify-subscribe`.

Output truncation in messaging

Gateway platforms have practical message-length caps. If `/kanban list`, `/kanban show`, or `/kanban tail` produce more than ~3800 characters of output, the response is truncated with a `... (truncated; use \hermes kanban ...` in your terminal for full output)`` footer. The CLI surface has no such cap.

Autocomplete

In the interactive CLI, typing `/kanban` and hitting Tab cycles through the built-in subcommand list (`list`, `ls`, `show`, `create`, `assign`, `link`, `unlink`, `claim`, `comment`, `complete`, `block`, `unblock`, `archive`, `tail`, `dispatch`, `context`, `init`, `gc`). The remaining verbs listed in the CLI reference above (`watch`, `stats`, `runs`, `log`, `assignees`, `heartbeat`, `notify-subscribe`, `notify-list`, `notify-unsubscribe`, `daemon`) also work — they're just not in the autocomplete hint list yet.

Collaboration patterns

The board supports these eight patterns without any new primitives:

Pattern	Shape	Example
P1 Fan-out	N siblings, same role	"research 5 angles in parallel"

P2 Pipeline	role chain: scout → editor → writer	daily brief assembly
P3 Voting / quorum	N siblings + 1 aggregator	3 researchers → 1 reviewer picks
P4 Long-running journal	same profile + shared dir + cron	Obsidian vault
P5 Human-in-the-loop	worker blocks → user comments → unblock	ambiguous decisions
P6 @mention	inline routing from prose	@reviewer look at this
P7 Thread-scoped workspace	/kanban here in a thread	per-project gateway threads
P8 Fleet farming	one profile, N subjects	50 social accounts
P9 Triage specifier	rough idea → triage → hermes kanban specify expands body → todo	"turn this one-liner into a spec'd task"

For worked examples of each, see docs/hermes-kanban-v1-spec.pdf.

Handing context to follow-up cards (the parent link)

A parent link is not just a scheduling gate — it is the context handoff channel from a **completed** card to a new one. When you create a card with `--parent <done-card-id>`, two things happen:

1. **It's immediately eligible.** `create_task` sets status by parent state: a child whose parents are all done is created directly in ready — no waiting, no manual promotion. (Children of still-open parents sit in `todo` until `recompute_ready` promotes them when the last parent finishes.)
2. **The parent's handoff rides along.** The worker context assembled for the child (`build_worker_context`, what `kanban_show()` returns) contains a `## Parent task results` section with each parent's completion summary and metadata, verbatim:

This is why the pattern for follow-up work on a finished card is **a new child card, not reopening the done card**. Completed cards are immutable history — their context flows forward through the parent link. Same-card rework (retry loops on a failing card) is a different mechanism: prior attempts on the *same* card surface as "prior attempts" in that card's own context.

A worktree or branch alone is not a substitute: repo state tells the follow-up worker *what* the code looks like, but not *why* — the decisions, tests run, and files touched live in the parent's structured handoff, not in git. Evidence that didn't exist when the parent completed (e.g. a CI log that failed later) belongs in the new card's **body**.

The remediation worker spawns with the original card's summary and metadata (changed files, decisions) already in context, plus the fresh evidence you put in the body.

Reconciling colliding worker branches

In engineering pipelines (P1/P2 with worktrees), two workers' branches can conflict when merged. Don't let either worker self-adjudicate — the

colliding agent lacks its peer's context and reliably overwrites the other side or abandons its own. Instead, create a reconciliation card assigned to a **third, neutral profile** with **both** conflicted cards linked as parents: the parent links carry both sides' completion summaries into the reconciler's context, so it receives both diffs *and* both intents. The bundled [merge-reconciler skill](#) gives that worker the full procedure: classify each conflicted hunk, resolve impartially, verify, and hand back a summary naming every decision.

Collision hotspots in parallel campaigns

In wide campaigns some files become collision magnets: many workers each add a little to the same file, nobody owns keeping it small, and it turns into the site of constant merge conflicts. The mitigation is a comment convention, not a new primitive. A worker that notices its diff keeps colliding with siblings in one file — or that a file it touches keeps appearing in other cards' recent comments — should not silently pile on. Instead it leaves a comment on its own card with a recognizable prefix:

```
hotspot: hermes_cli/kanban_db.py – third conflicting edit to the dispatch loo
```

and repeats the flag in its completion metadata. Orchestrators (or humans reviewing the board) who see **two or more hotspot: comments naming the same path** should create a dedicated refactor/decomposition card for that file **before** queuing more work that touches it — splitting the magnet file is cheaper than reconciling every future collision it would cause. For conflicts that have *already* happened, use the reconciliation-card pattern above with the [merge-reconciler skill](#); hotspot flagging is the upstream fix that keeps the reconciler from becoming a standing lane.

Multi-tenant usage

When one specialist fleet serves multiple businesses, tag each task with a tenant:

Workers receive `$HERMES_TENANT` and namespace their memory writes by prefix. The board, the dispatcher, and the profile definitions are all shared; only the data is scoped.

Gateway notifications

When you run `/kanban create ...` from the gateway (Telegram, Discord, Slack, etc.), the originating chat is automatically subscribed to the new task. The gateway's background notifier polls `task_events` every few seconds and delivers one message per terminal event (completed

seconds and delivers one message per terminal event (completed, blocked, gave_up, crashed, timed_out) to that chat. Completed tasks also send the first line of the worker's `--result` so you see the outcome without having to `/kanban show`.

You can manage subscriptions explicitly from the CLI — useful when a script / cron job wants to notify a chat it didn't originate from:

A subscription removes itself automatically once the task reaches done or archived; no cleanup needed.

Multi-profile setups: delivery is profile-owned

In a one-gateway-per-profile deployment (one dispatcher, separate gateway processes for writer, admin, etc. — see the [multi-gateway guide](#)), dispatch and delivery have separate owners:

- **Dispatch stays single-owner.** Exactly one gateway keeps `kanban.dispatch_in_gateway: true` and runs the dispatcher; every other gateway sets it to `false`.
- **Notification delivery is profile-owned.** Every gateway — including non-dispatch ones — runs the notifier and polls only subscriptions stamped with a profile whose platform adapters it hosts. A task created from the writer profile's Telegram gets its `completed/blocked` message delivered by the writer gateway, even though the default gateway did the dispatching.
- **Legacy subscriptions** created before profile stamping (no `notifier_profile` on the row) are delivered only by the gateway that holds the actual dispatcher singleton lock, so two gateways never race for them.

Duplicate delivery across gateways is prevented by the atomic per-event claim in the board DB. No relays, credential sharing, or extra dispatchers are needed — each profile gateway simply delivers through its own adapters.

Runs — one row per attempt

A task is a logical unit of work; a **run** is one attempt to execute it. When the dispatcher claims a ready task it creates a row in `task_runs` and points `tasks.current_run_id` at it. When that attempt ends — completed, blocked, crashed, timed out, spawn-failed, reclaimed — the run row closes with an outcome and the task's pointer clears. A task that's been attempted three times has three `task_runs` rows.

Why two tables instead of just mutating the task: you need **full attempt history** for real-world postmortems ("the second reviewer attempt got to approve, the third merged"), and you need a clean place to hang per-attempt metadata — which files changed, which tests ran, which findings a reviewer noted. Those are run facts, not task facts.

Runs are also where **structured handoff** lives. When a worker completes a task (via `kanban_complete(...)`) it can pass:

- `summary` (tool param) / `--summary` (CLI) — human handoff; goes on the run; downstream children see it in their `build_worker_context`.
- `metadata` (tool param) / `--metadata` (CLI) — free-form JSON dict on the run; children see it serialized alongside the summary.
- `result` (tool param) / `--result` (CLI) — short log line that goes on the task row (legacy field, kept for back-compat).

Downstream children read the most recent completed run's summary + metadata for each parent. Retrying workers read the prior attempts on their own task (outcome, summary, error) so they don't repeat a path that already failed.

The same handoff is reachable from the CLI when you (the human) need to close out a task a worker can't — e.g. a task that was abandoned, or one you marked done manually from the dashboard:

Runs are exposed on the dashboard (Run History section in the drawer, one coloured row per attempt) and on the REST API (`GET /api/plugins/kanban/tasks/:id` returns a `runs[]` array). `PATCH /api/plugins/kanban/tasks/:id` with `{status: "done", summary, metadata}` forwards both to the kernel, so the dashboard's "mark done" button is CLI-equivalent. `task_events` rows carry the `run_id` they belong to so the UI can group them by attempt, and the `completed` event embeds the first-line summary in its payload (capped at 400 chars) so gateway notifiers can render structured handoffs without a second SQL round-trip.

Bulk close caveat. `hermes kanban complete a b c --summary X` is refused — structured handoff is per-run, so copy-pasting the same summary to N tasks is almost always wrong. Bulk close *without* `--summary` / `--metadata` still works for the common "I finished a pile of admin tasks" case.

Reclaimed runs from status changes. If you drag a running task off running in the dashboard (back to ready, or straight to todo), or archive a task that was still running, the in-flight run closes with `outcome='reclaimed'` rather than being orphaned. The `task_runs` row is always in a terminal state when `tasks.current_run_id` is NULL, and vice versa — that invariant holds across CLI, dashboard, dispatcher, and notifier.

Synthetic runs for never-claimed completions. Completing or blocking a task that was never claimed (e.g. a human closes a ready task from the dashboard with a summary, or a CLI user runs `hermes kanban complete <ready-task> --summary X`) would otherwise drop the handoff. Instead the kernel inserts a zero-duration run row (`started_at == ended_at`) carrying the summary / metadata / reason so attempt history stays complete. The `completed` / `blocked` event's `run_id` points at that row.

Live drawer refresh. When the dashboard's WebSocket event stream reports new events for the task the user is currently viewing, the drawer reloads itself (via a per-task event counter threaded into its `useEffect` dependency list). Closing and reopening is no longer required to see a run's new row or updated outcome.

Forward compatibility

Two nullable columns on tasks are reserved for v2 workflow routing: `workflow_template_id` (which template this task belongs to) and `current_step_key` (which step in that template is active). The v1 kernel ignores them for routing but lets clients write them, so a v2 release can add the routing machinery without another schema migration.

Event reference

Every transition appends a row to `task_events`. Each row carries an optional `run_id` so UIs can group events by attempt. Kinds group into three clusters so filtering is easy (`hermes kanban watch --kinds completed,gave_up,timed_out`):

Lifecycle (what changed about the task as a logical unit):

Kind	Payload	When
created	{assignee, status, parents, tenant}	Task inserted. <code>run_id</code> is NULL.
promoted	—	<code>todo</code> → <code>ready</code> because all parents hit done. <code>run_id</code> is NULL.
claimed	{lock, expires, run_id}	Dispatcher atomically claimed a <code>ready</code> task for spawn.
completed	{result_len, summary?}	Worker wrote <code>--result</code> / <code>--summary</code> and task hit done. <code>summary</code> is the first-line handoff (400-char cap); full version lives on the run row. If <code>complete_task</code> is called on a never-claimed task with handoff fields, a zero-duration run is synthesized so <code>run_id</code> still points at something.
blocked	{reason, kind, recurrences}	Worker or human flipped the task to blocked. <code>kind</code> is the typed block reason (<code>needs_input</code> , <code>capability</code> , <code>transient</code> , or null for a generic block); <code>recurrences</code> is the unblock-loop counter. Synthesizes a zero-duration run when called on a never-claimed task with <code>--reason</code> .
dependency_wait	{reason, kind}	Worker blocked with <code>kind=dependency</code> — the task is only waiting on another task, so it routes to <code>todo</code> (parent-gated, auto-promoted) instead of <code>blocked</code> . No human needed.
block loop detected	{reason, kind,	A task was unblocked and re-blocked for the same reason BLOCK_RECURRENCE_LIMIT times (default 2). Instead of landing in <code>blocked</code> again —

block_loop_detected	recurrences, limit}	where a cron would keep unblocking it — it routes to triage for a human decision, breaking the unblock↔re-block loop.
unblocked	—	blocked → ready (or todo if parents are still open), either manually or via /unblock. Resets the dispatcher's consecutive_failures but deliberately preserves block_recurrences so the loop breaker keeps its memory. run_id is NULL.
archived	—	Hidden from the default board. If the task was still running, carries the run_id of the run that was reclaimed as a side effect.

Edits (human-driven changes that aren't transitions):

Kind	Payload	When
assigned	{assignee}	Assignee changed (including unassignment).
edited	{fields}	Title or body updated.
reprioritized	{priority}	Priority changed.
status	{status}	Dashboard drag-drop wrote a status directly (e.g. todo → ready). Carries the run_id of the run that was reclaimed when dragging off running; otherwise run_id is NULL.

Worker telemetry (about the execution process, not the logical task):

Kind	Payload	When
spawned	{pid}	Dispatcher successfully started worker process.
heartbeat	{note?}	Worker called hermes kanban he \$TASK to signal liveness during local operations.
reclaimed	{stale_lock}	Claim TTL expired without a con task goes back to ready.
crashed	{pid, claimer}	Worker PID no longer alive but T hadn't expired yet.
timed_out	{pid, elapsed_seconds, limit_seconds, sigkill}	max_runtime_seconds exceeded; dispatcher SIGTERM'd (then SIG after 5 s grace) and re-queued.
stale	{elapsed_seconds, last_heartbeat_at, heartbeat_age_seconds, timeout_seconds, pid, terminated}	Task ran longer than kanban.dispatch_stale_timeout (default 4 h) AND no kanban_heartbeat arrived in the last hour. Dispatcher SIGTERM'd the host-local worker (any), reset the task to ready for dispatch. Does NOT tick the failure counter (stale is dispatcher-side absence detection, not a worker failure). Workers running long operations: call kanban_heartbeat at least once an hour to avoid this.
orphaned	{reason, claim_lock, claim_expires}	Orphaned-card reconciliation: task was running with broken claim bookkeeping (claim_lock or claim_expires NULL — crash mitigation: manual SQL, DB restore) and no worker seen since the TTL expired.

reconciled	<code>claim_expires,</code> <code>worker_pid</code>	worker, so none of the TTL/crash paths could ever recover it. The dispatcher requeued it to ready explanatory comment. Gated by <code>kanban.reconcile_orphans</code> in <code>config.yaml</code> (default <code>true</code>).
respawn_guarded	<code>{reason}</code>	Dispatcher refused to re-spawn ready task this tick. Reasons: <code>blocker_auth</code> (last failure was a <code>quota/auth/429</code> error — wait for window to reset), <code>recent_success</code> completed run happened in the hour — wait for review before re-running), <code>active_pr</code> (a GitHub PR appears in a recent comment — worker already opened a PR). Task stays in <code>ready</code> ; the next tick gets chance to spawn. If the underlying condition persists, the normal <code>consecutive_failures</code> circuit breaker will auto-block via <code>gave_up</code> after <code>failure_limit</code> failures.
spawn_failed	<code>{error, failures}</code>	One spawn attempt failed (missing PATH, workspace unmountable, Counter increments; task return ready for retry.
protocol_violation	<code>{pid, claimer,</code> <code>exit_code,</code> <code>protocol_violation}</code>	Worker exited successfully while task was still running, usually because answered without calling <code>kanban_complete</code> or <code>kanban_block</code> . Emitted on every violation (the payload's <code>protocol_violation:marker</code> is copied into the run metadata and feeds the violation-only retry budget). Below the budget — up to <code>_PROTOCOL_VIOLATION_FAILURE_LIMIT</code> (default 3) <i>consecutive</i> violation task <code>max_retries</code> overriding — task simply returns to <code>ready</code> for another attempt; when the streak reaches bound the dispatcher also emits and auto-blocks.
gave_up	<code>{failures,</code> <code>effective_limit,</code> <code>limit_source, error}</code>	Circuit breaker fired after N consecutive non-successful attempts. Task is blocked with the last error. The effective limit resolves as <code>task_max_retries</code> / <code>dispatcher_failure_limit</code> / <code>kanban.failure_limit</code> , then the default.

`hermes kanban tail <id>` shows these for a single task. `hermes kanban watch` streams them board-wide.

Out of scope

Kanban is deliberately single-host. `~/hermes/kanban.db` is a local SQLite file and the dispatcher spawns workers on the same machine. Running a shared board across two hosts is not supported — there's no coordination primitive for "worker X on host A, worker Y on host B," and the crash-detection path assumes PIDs are host-local. If you need multi-host, run an independent board per host and use a distributed database to aggregate events to

independent board per nost and use `delegate_task` / a message queue to bridge them.

Design spec

The complete design — architecture, concurrency correctness, comparison with other systems, implementation plan, risks, open questions — lives in `docs/hermes-kanban-v1-spec.pdf`. Read that before filing any behavior-change PR.